Pregunta 1

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Vectores - Ejercicio 1 (Básico)

Implementar la función `sumarVector` que recibe un vector de enteros `vec` y la cantidad de elementos `n`, y devuelve la **suma total de sus elementos**.

Indicaciones: Si `n == 0`, debe retornar `0`.

Firma requerida:

```
int sumarVector(int vec[], int n);
```

Por ejemplo:

Prueba	Resultado
`int v[] = {10, 20, 30, 40};` `cout << sumarVector(v, 4);`	`100`

Respuesta:
(sistema de penalización: 0 %)

Reiniciar respuesta

```
int sumarVector(int vec[], int n) {
  int suma = 0;
  for(int i = 0; i < n; i++){
      suma += vec[i];
  }
return suma;

}
```

	Prueba	Esperado	Conseguido	
✓	`int v[] = {10, 20, 30, 40};` `cout << sumarVector(v, 4);`	`100`	`100`	✓
✓	`int v[] = {-5, 5, -10, 10, 2};` `cout << sumarVector(v, 5);`	`2`	`2`	✓
✓	`int v[] = {};` `cout << sumarVector(v, 0);`	`0`	`0`	✓

Todas las pruebas superadas.

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 2

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Vectores - Ejercicio 2 (Básico)

Implementar la función `contarPares` que recibe un vector de enteros `vec` y la cantidad de elementos `n`, y devuelve la **cantidad de números pares** contenidos en el vector.

Indicaciones: Un número es par si el resto de su división por 2 es cero (`x % 2 == 0`). Si `n == 0`, debe retornar `0`.

Firma requerida:

```
int contarPares(int vec[], int n);
```

Por ejemplo:

Prueba	Resultado
`int v[] = {1, 2, 3, 4, 5, 6};` `cout << contarPares(v, 6);`	3

Respuesta:
(sistema de penalización: 0 %)

Reiniciar respuesta

```
1  int contarPares(int vec[], int n) {
2      int cantidad = 0;
3      for(int i=0;i<n;i++){
4          if(vec[i] % 2 == 0){
5              cantidad ++;
6          }
7
8
9      }
10     return cantidad;
11 }
```

	Prueba	Esperado	Conseguido	
✓	`int v[] = {1, 2, 3, 4, 5, 6};` `cout << contarPares(v, 6);`	3	3	✓
✓	`int v[] = {0, -2, -4, 8};` `cout << contarPares(v, 4);`	4	4	✓
✓	`int v[] = {};` `cout << contarPares(v, 0);`	0	0	✓

Todas las pruebas superadas.

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 3

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Vectores - Ejercicio 3 (Básico / Intermedio)

Implementar la función `buscarPosicionMaximo` que recibe un vector de enteros `vec` y la cantidad de elementos `n`, y devuelve el **índice (posición) donde se encuentra el valor máximo**.

Indicaciones:

- Si hay valores máximos duplicados, debe retornar el índice de la **primera aparición**.
- Si el vector está vacío (`n == 0`), debe retornar `-1` .

Firma requerida:

```
int buscarPosicionMaximo(int vec[], int n);
```

Por ejemplo:

Prueba	Resultado
`int v[] = {5, 12, 8, 20, 3};` `cout << buscarPosicionMaximo(v, 5);`	3

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
int buscarPosicionMaximo(int vec[], int n) {
    if (n == 0){
        return -1;
    }

    int posMax = 0;
    for (int i=0;i<n;i++){
        if(vec[i] > vec[posMax]){
            posMax = i;
        }
    }
    return posMax;
}
```

	Prueba	Esperado	Conseguido	
✓	`int v[] = {15, 2, 15, 4};` `cout << buscarPosicionMaximo(v, 4);`	0	0	✓
✓	`int v[] = {-10, -5, -20};` `cout << buscarPosicionMaximo(v, 3);`	1	1	✓

✓	`int v[] = {};` `cout << buscarPosicionMaximo(v, 0);`	`-1`	`-1`	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 4

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Vectores - Ejercicio 4 (Intermedio)

Implementar la función `contarMayoresAlPromedio` que recibe un vector de números flotantes (`float vec[]`) y su cantidad de elementos `n` . Debe calcular el promedio de los elementos y devolver la **cantidad de elementos que son estrictamente mayores a dicho promedio**.

Indicaciones: Si `n == 0` , debe retornar `0` .

Firma requerida:

```
int contarMayoresAlPromedio(float vec[], int n);
```

Por ejemplo:

Prueba	Resultado

`float v[] = {2.0, 4.0, 6.0, 8.0}; // Promedio = 5.0`
`cout << contarMayoresAlPromedio(v, 4);`
	2

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
int contarMayoresAlPromedio(float vec[], int n) {
    if (n==0){
        return 0;
    }

    float suma = 0.0f;
    for(int i=0;i<n;i++){
        suma += vec[i];
    }
    float promedio = suma / n;
    int contador = 0;

    for(int i=0;i<n;i++){
        if(vec[i] > promedio){
           contador ++;
        }
    }
return contador;

}
```

	Prueba	Esperado	Conseguido	

✓	`float v[] = {2.0, 4.0, 6.0, 8.0}; // Promedio = 5.0`
	`cout << contarMayoresAlPromedio(v, 4);`
		2	2	✓

✓	`float v[] = {10.0, 10.0, 10.0}; // Promedio = 10.0`
	`cout << contarMayoresAlPromedio(v, 3);`
		`0`	`0`	✓

	Prueba	Esperado	Conseguido	
✓	`float v[] = {};` `cout << contarMayoresAlPromedio(v, 0);`	`0`	`0`	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 5

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Vectores - Ejercicio 5 (Intermedio - Modificación In-Place)

Implementar el procedimiento `invertirVector` que recibe un vector de enteros `vec` y su cantidad de elementos `n`, y **invierte el orden de sus elementos directamente en el vector** (in-place).

Indicaciones: El primer elemento pasa a ser el último, el segundo el penúltimo, etc. No debe retornar ningún valor (usar `void`).

Firma requerida:

```
void invertirVector(int vec[], int n);
```

Por ejemplo:

Prueba	Resultado

`int v[] = {1, 2, 3, 4, 5};`
`invertirVector(v, 5);`
`for(int i=0; i<5; i++) cout << v[i];`
	54321

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
void invertirVector(int vec[], int n) {
    for(int i = 0;i<n / 2; i++){
        int temp= vec[i];
        vec[i] = vec[n-1-i];
        vec[n-1-i]=temp;
    }
}
```

	Prueba	Esperado	Conseguido	

✓	`int v[] = {1, 2, 3, 4, 5};`
	`invertirVector(v, 5);`
	`for(int i=0; i<5; i++) cout << v[i];`
		54321	54321	✓

✓	`int v[] = {10, 20};` `invertirVector(v, 2);`	2010	2010	✓

	`for(int i=0; i<2; i++) cout << v[i];`			

Todas las pruebas superadas.

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 6

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Matrices - Ejercicio 6 (Básico / Intermedio)

Se cuenta con las constantes globales `MAX_COLS = 100` y `MAX_FILAS = 100` ya definidas.

Implementar la función `sumarMatriz` que recibe una matriz bidimensional de enteros `mat`, la cantidad de `filas` y la cantidad de columnas reales ocupadas `cols`, y devuelve la **suma de todos los elementos de la matriz**.

Firma requerida:

```
int sumarMatriz(int mat[][MAX_COLS], int filas, int cols);
```

Por ejemplo:

Prueba	Resultado
`int m[MAX_FILAS][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}};` `cout << sumarMatriz(m, 2, 3);`	21

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
1  const int MAX_COLS = 100;
2  const int MAX_FILAS = 100;
3
4  int sumarMatriz(int mat[][MAX_COLS], int filas, int cols) {
5      int suma = 0;
6      for (int i= 0; i< filas;i++){
7          for(int j=0;j<cols;j++){
8              suma +=mat[i][j];
9          }
10     }
11     return suma;
12 }
```

	Prueba	Esperado	Conseg
✓	`int m[MAX_FILAS][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}};` `cout << sumarMatriz(m, 2, 3);`	21	21
✓	`int m[MAX_FILAS][MAX_COLS] = {{10, -5}, {20, -10}, {2, 3}};` `cout << sumarMatriz(m, 3, 2);`	20	20
✓	`int m[MAX_FILAS][MAX_COLS] = {{5}};` `cout << sumarMatriz(m, 1, 1);`	5	5

Todas las pruebas superadas.

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 7

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Matrices - Ejercicio 7 (Intermedio - Diagonal Principal)

Se cuenta con las constantes globales `MAX_COLS = 100` y `MAX_FILAS = 100` ya definidas.

Implementar la función `sumarDiagonalPrincipal` que recibe una matriz cuadrada de enteros `mat` y su dimensión `n` (es decir, una matriz de `n` filas por `n` columnas), y devuelve la **suma de los elementos pertenecientes a la diagonal principal** (elementos donde fila == columna).

Firma requerida:

```
int sumarDiagonalPrincipal(int mat[][MAX_COLS], int n);
```

Por ejemplo:

Prueba	Resultado
`int m[100][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};` `cout << sumarDiagonalPrincipal(m, 3);`	15

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
1  const int MAX_COLS = 100;
2  const int MAX_FILAS = 100;
3
4  int sumarDiagonalPrincipal(int mat[][MAX_COLS], int n) {
5      int suma =0;
6      for(int i=0;i<n;i++){
7          suma+=mat[i][i];
8      }
9
10     return suma;
11 }
```

	Prueba	Esperado	Consegui
✓	`int m[100][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};` `cout << sumarDiagonalPrincipal(m, 3);`	15	15
✓	`int m[100][MAX_COLS] = {{10, 0}, {0, 20}};` `cout << sumarDiagonalPrincipal(m, 2);`	30	30
✓	`int m[100][MAX_COLS] = {{-4}};` `cout << sumarDiagonalPrincipal(m, 1);`	-4	-4

Todas las pruebas superadas.

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 8

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Matrices y Vectores - Ejercicio 8 (Intermedio / Avanzado)

Se cuenta con las constantes globales `MAX_COLS = 100` y `MAX_FILAS = 100` ya definidas.

Implementar el procedimiento `obtenerMaximosPorFila` que recibe una matriz de enteros `mat`, sus dimensiones `filas` y `cols`, y un vector `maximos[]`. El procedimiento debe encontrar el **mayor elemento de cada fila** de la matriz y almacenarlo en la posición correspondiente del vector `maximos`.

Firma requerida:

```
void obtenerMaximosPorFila(int mat[][MAX_COLS], int filas, int cols, int maximos[]);
```

Por ejemplo:

Prueba	Resultado

`int m[100][MAX_COLS] = {{3, 8, 1}, {9, 2, 5}, {4, 4, 4}};`
`int maximosFilas[3];`
`obtenerMaximosPorFila(m, 3, 3, maximosFilas);`
`for(int i=0; i<3; i++) cout << maximosFilas[i] << " ";`
	`8 9 4`

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
1  const int MAX_COLS = 100;
2  const int MAX_FILAS = 100;
3
4  void obtenerMaximosPorFila(int mat[][MAX_COLS], int filas, int cols, int maximos[
5      for (int i=0;i<filas;i++){
6           int maxFila = mat[i][0];
7          for(int j=1;j < cols; j++){
8              if(mat[i][j]>maxFila){
9              maxFila=mat[i][j];
10         }
11         }
12         maximos[i]=maxFila;
13     }
14
15 }
```

	Prueba	Esperado	Consegui

✓	`int m[100][MAX_COLS] = {{3, 8, 1}, {9, 2, 5}, {4, 4, 4}};`
	`int maximosFilas[3];`
	`obtenerMaximosPorFila(m, 3, 3, maximosFilas);`
	`for(int i=0; i<3; i++) cout << maximosFilas[i] << " ";`
		`8 9 4`	`8 9 4`

	Prueba	Esperado	Conseguí

✓	`int m[100][MAX_COLS] = {{-10, -5}, {-1, -20}};`
	`int maximosFilas[2];`
	`obtenerMaximosPorFila(m, 2, 2, maximosFilas);`
	`for(int i=0; i<2; i++) cout << maximosFilas[i] << " ";`
		`-5 -1`	`-5 -1`

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 9

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Matrices - Ejercicio 9 (Avanzado - Transpuesta)

Se cuenta con las constantes globales `MAX_COLS = 100` y `MAX_FILAS = 100` ya definidas.

Implementar el procedimiento `transponerMatriz` que recibe una matriz `origen` de dimensiones `filas` X `cols` y carga en la matriz `destino` de dimensiones `cols` X `filas` su **matriz transpuesta** (es decir, donde los elementos de la fila `i` y columna `j` de origen pasan a la fila `j` y columna `i` de destino: `destino[j][i] = origen[i][j]`).

Firma requerida:

```
void transponerMatriz(int origen[][MAX_COLS], int filas, int cols, int destino[][MAX_COLS]);
```

Por ejemplo:

Prueba	Resultado

`int orig[100][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}};`
`int dest[100][MAX_COLS];`
`transponerMatriz(orig, 2, 3, dest);`
`for(int i=0; i<3; i++) {`
`  for(int j=0; j<2; j++) cout << dest[i][j] << " ";`
`  cout << endl;`
`}`
	`1 4`
	`2 5`
	`3 6`

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
1  const int MAX_COLS = 100;
2  const int MAX_FILAS = 100;
3
4  void transponerMatriz(int origen[][MAX_COLS], int filas, int cols, int destino[][
5      for (int i= 0; i < filas; i++){
6          for(int j=0;j<cols;j++){
7              destino[j][i]=origen[i][j];
8          }
9      }
10 }
```

	Prueba	Esperado	Conseguido

✓	`int orig[100][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}};`
	`int dest[100][MAX_COLS];`
	`transponerMatriz(orig, 2, 3, dest);`
	`for(int i=0; i<3; i++) {`
	`  for(int j=0; j<2; j++) cout << dest[i][j] << " ";`
	`  cout << endl;`
	`}`
		1 4
		2 5
		3 6
			1 4
			2 5
			3 6

✓	`int orig[100][MAX_COLS] = {{7, 8}, {9, 10}};`
	`int dest[100][MAX_COLS];`
	`transponerMatriz(orig, 2, 2, dest);`
	`for(int i=0; i<2; i++) {`
	`  for(int j=0; j<2; j++) cout << dest[i][j] << " ";`
	`  cout << endl;`
	`}`
		7 9
		8 10
			7 9
			8 10

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 1,00/1,00.

Pregunta 10

Marcar pregunta

Correcta Se puntúa 1,00 sobre 1,00

Unidad 7: Vectores y Estructuras de Datos

Se cuenta con la siguiente estructura ya definida en el sistema:

```
struct Alumno {
    int legajo;
    string nombre;
    float nota1;
    float nota2;
};
```

Consigna:

Implementar la función `legajoMejorPromedio`, la cual recibe como parámetros un **vector de estructuras** `Alumno` y un número entero `n` que representa la **cantidad de alumnos** cargados en el vector.

La función debe calcular el promedio de notas *((nota1 + nota2) / 2)* de cada alumno y **devolver el legajo del alumno que posee el mayor promedio**.

Precondiciones e indicaciones:

- Si el vector está vacío (`n == 0`), la función debe retornar `-1` .
- En caso de empate en el promedio máximo, debe retornar el legajo del primer alumno encontrado con dicho promedio.
- **No** debes incluir la definición de la estructura ni la función `main` en tu respuesta, únicamente el código de la función solicitada.

Firma requerida:

```
int legajoMejorPromedio(Alumno alumnos[], int n);
```

Por ejemplo:

Prueba	Resultado

`Alumno curso[] = {`
`  {1001, "Juan Perez", 7.0, 8.0},  // Promedio: 7.5`
`  {1002, "Maria Gomez", 9.0, 10.0},  // Promedio: 9.5`
`  {1003, "Carlos Lopez", 6.0, 6.0}  // Promedio: 6.0`
`};`
`cout << legajoMejorPromedio(curso, 3);`
	1002

Respuesta:

(sistema de penalización: 0 %)

Reiniciar respuesta

```
struct Alumno {
    int legajo;
    string nombre;
    float nota1;
    float nota2;
};

int legajoMejorPromedio(Alumno alumnos[], int n) {
    if(n==0){
```

```
        return -1;
    }
    int mejorLegajo = alumnos[0].legajo;
    float mejorPromedio = (alumnos[0].nota1 + alumnos[0].nota2) / 2.0f;

    for (int i=0;i<n;i++){
        float promedioActual = (alumnos[i].nota1 + alumnos[i].nota2)/2.0f;
        if(promedioActual>mejorPromedio){
            mejorPromedio = promedioActual;
            mejorLegajo = alumnos[i].legajo;
        }
    }
    return mejorLegajo;
}
```

	Prueba	Esperado	Co

✓	`Alumno curso[] = {`
	`  {1001, "Juan Perez", 7.0, 8.0},  // Promedio: 7.5`
	`  {1002, "Maria Gomez", 9.0, 10.0},  // Promedio: 9.5`
	`  {1003, "Carlos Lopez", 6.0, 6.0}  // Promedio: 6.0`
	`};`
	`cout << legajoMejorPromedio(curso, 3);`
		1002	100

✓	`Alumno curso[] = {};` `cout << legajoMejorPromedio(curso, 0);`	-1	-1

✓	`Alumno curso[] = {`
	`  {2001, "Pedro Sanchez", 8.0, 8.0}, // Promedio: 8.0 (Primero)`
	`  {2002, "Laura Torres", 8.0, 8.0},  // Promedio: 8.0`
	`  {2003, "Sofia Diaz", 5.0, 7.0}  // Promedio: 6.0`
	`};`
	`cout << legajoMejorPromedio(curso, 3);`
		2001	200

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 1,00/1,00.

Finalizar revisión